

Operating Systems

CSC 309

Lecture Notes

COmputer Science

UNILag

24/25

Prepared by
Moyo

Contents

Chapter 2: Process Management	3
Chapter 3: Memory Management	4
Chapter 6: Case Studies and Laboratory Practice	7

Chapter 2

Process Management

User Safety: safe

Chapter 3

Memory Management

3.1 Memory Management

Memory management is the core responsibility of an operating system that involves allocating, tracking, and reclaiming main memory for processes. This chapter introduces the concepts of virtual memory and the mechanisms that support it.

The abstraction of virtual memory allows each process to view the available memory as a contiguous address space, independent of the physical RAM installed in the machine. The operating system maintains data structures that translate virtual addresses used by programs into physical addresses in RAM or, when necessary, into secondary storage.

3.1.1 Virtual Memory Concepts

Virtual memory provides isolation and protection between processes while extending the address space beyond the limits of physical memory. It enables features such as memory-mapped files and on-demand loading of code and data.

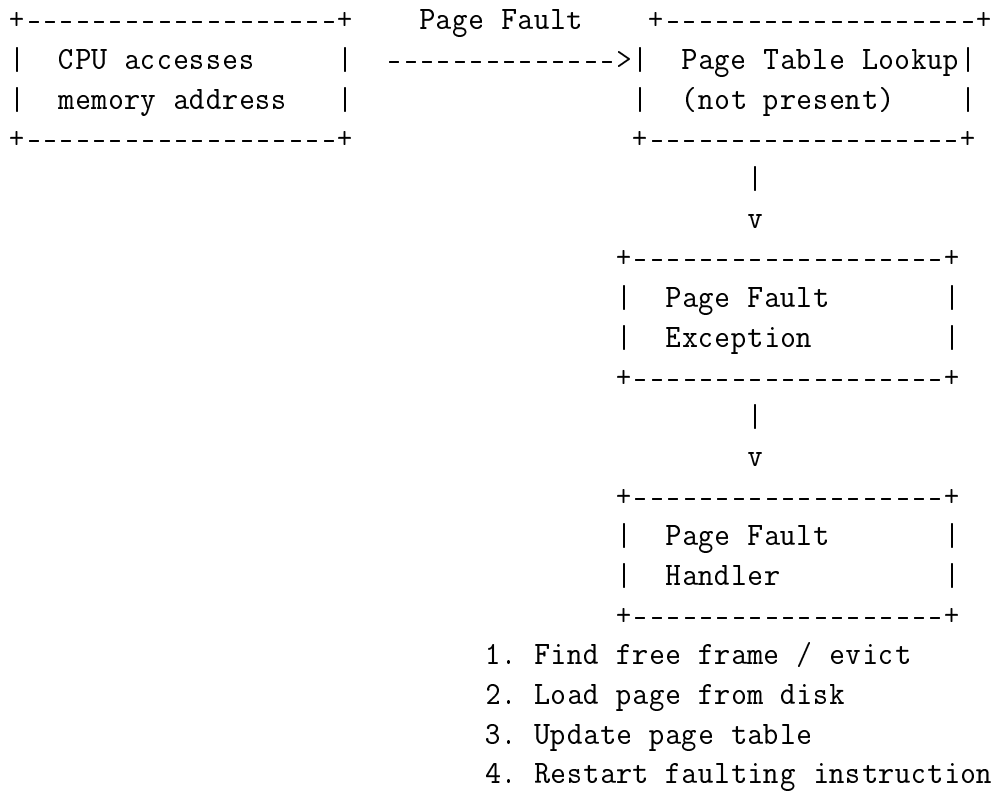
3.1.2 Paging Mechanism

Paging divides both virtual and physical memory into fixed-size blocks called pages and frames respectively. The operating system uses a page table to keep track of the mapping between a process's pages and the available frames.

+-----+	+-----+
Process A	Physical Memory
Code Data Heap	Frame 0 Frame 1
+-----+	+-----+
Process B	Frame 2 Frame 3
+-----+	+-----+
Free Frames	...
+-----+	

3.1.3 Page Fault Handling

When a process accesses a page that is not resident in physical memory, a page fault occurs. The operating system handles the fault by allocating a frame, reading the required page from disk, updating the page table, and restarting the interrupted instruction.



3.1.4 Comparison of Physical and Virtual Memory

Aspect	Description
Physical Memory	The actual RAM installed in the computer; limited in size and directly accessible by the processor.
Virtual Memory	An abstraction that provides each process with an address space larger than physical memory.

Table 3.1: Comparison of Physical and Virtual Memory

3.1.5 Swap Space

Swap space is a designated area on secondary storage that the operating system uses to store pages that are not currently in physical memory. It allows the system to overcommit memory and to free frames for other processes.

3.1.6 Summary

Memory management combines allocation strategies, protection mechanisms, and swapping policies to present a coherent virtual address space to processes. The next chapter will explore how files are organized and accessed through the file system and I/O subsystem.

Tutorial Questions

1. Explain the difference between physical memory and virtual memory. Provide one advantage of using virtual memory for process isolation.
2. Describe how a page table maps virtual pages to physical frames. Include a brief example with two processes.
3. What is a page fault and what steps does the operating system take to resolve it?
4. Compare the concepts of a frame and a page. Why must they be of the same size?
5. Discuss the purpose of swap space. How does it affect system performance?
6. A process attempts to access a page that is not in memory, causing a page fault. Explain the sequence of events that follow, from detection to resumption of the process.
7. Identify two scenarios where demand paging improves overall system throughput.
8. Outline the conditions under which a page replacement algorithm must be invoked.

Chapter 6

Case Studies and Laboratory Practice

6.1 Case Studies

A **Case Study** is a detailed examination of a specific instance that illustrates broader operating system concepts. Case studies enable students to analyze real-world scenarios, identify underlying mechanisms, and draw conclusions about system behavior. They are particularly useful for exploring complex interactions that are difficult to isolate in controlled experiments.

6.2 Practical Laboratory Work

The term **Laboratory** refers to a structured **Hands-on** activity that combines guided instructions with practical tasks. This chapter includes **Practical Lab Work** that focuses on Linux-based experiments and simulation projects. Each **Exercise** is designed to reinforce understanding of a specific concept or technique.

6.2.1 Lab Workflow

Setup -> Execute -> Collect -> Analyze & Report

6.2.2 Hands-on Linux Exercises

The following exercise guides students through the configuration of CPU Scheduling policies on a Linux system and the collection of performance metrics.

```
# Sample script to run a round-robin benchmark
taskset -c 0 ./benchmark.sh &
for i in {1..5}; do
    taskset -c 0 ./benchmark.sh &
done
wait
```

6.3 Simulation of Scheduling and Synchronization Algorithms

The **Simulation** component models the behavior of scheduling and synchronization mechanisms using a computational framework. This allows exploration of scenarios that are difficult or impractical to reproduce on physical hardware.

6.3.1 Simulation Framework

The framework captures system state, processes events, and records outcomes. It supports both **Preemptive Scheduling** and **Non-preemptive Scheduling**, as well as **Mutex** and **Semaphore** synchronization.

```
+-----+ +-----+ +-----+ +-----+ +-----+
| New   |-->| Ready |-->| Running|-->| Waiting|-->| Terminated |
+-----+ +-----+ +-----+ +-----+ +-----+
```

Preemptive Scheduling:

```
+-----+
| CPU running P |
+-----+
      |
      v
+-----+
| Timer interrupt|
+-----+
      |
      v
+-----+
| Scheduler selects Q |
+-----+
```

Non-preemptive Scheduling:

```
+-----+
| CPU running P |
+-----+
      |
      v
+-----+
| P voluntarily yields |
+-----+
      |
      v
+-----+
```



```

| Scheduler selects Q |
+-----+

```

Simulation Process

Simulation Process

```

|
v
+-----+
| Load Program      |
+-----+
|
v
+-----+
| Execute Instructions|
+-----+
|
v
+-----+
| Record Events      |
+-----+
|
v
+-----+
| Analyze Results    |
+-----+
|
v
+-----+
| Generate Report    |
+-----+

```

Example: Round-Robin Scheduling Simulation

```

# Pseudo-code for a round-robin simulator
processes = [P1, P2, P3, P4]
quantum   = 4
ready_q   = queue(processes)

while not ready_q.empty():
    p = ready_q.dequeue()
    execute(p, min(quantum, p.remaining_time))
    if p.remaining_time > quantum:
        p.remaining_time -= quantum

```

```

        ready_q.enqueue(p)
    else:
        p.remaining_time = 0
        p.state = TERMINATED
        record(p)

```

[P1] -> [P2] -> [P3] -> [P4] -> (loop)

Approach	Key Characteristics
Case Study	In-depth analysis of a single instance; focuses on context and detailed observation.
Laboratory	Controlled setting for repeated experimentation; emphasizes hands-on execution.
Simulation	Abstract model that mimics system behavior; enables exploration beyond physical limits.

6.4 Conclusion

The practical experiences described in this chapter provide a solid foundation for further research in system optimization and advanced operating system design. The techniques and methodologies introduced here will be built upon in future studies involving performance modeling and system verification.

Tutorial Questions

1. Describe the purpose of a case study in operating systems education. Provide an example from this chapter.
2. Explain the steps involved in conducting a hands-on Linux exercise to evaluate CPU Scheduling.
3. Compare two scheduling algorithms using a simulation scenario. Which algorithm performed better under high load and why?
4. Identify a common race condition that can arise during a synchronization simulation and propose a synchronization primitive to prevent it.
5. Explain how a monitor can be used to enforce mutual exclusion in a simulated critical section.
6. Discuss the role of a lab report in documenting the results of a scheduling simulation. List its essential sections.
7. Provide an example of a practical lab exercise that investigates virtual memory paging. What metrics would you measure?

8. Explain the difference between preemptive and non-preemptive scheduling in the context of a simulation. How does each affect the simulation logic?